

PIE TIME

[PicoCTF '25] PIE TIME

This challenge appears to be just like many intro level binary exploitation CTF challenges: Find the address of a function in the binary and enter it in the program at which point the flag will be printed! The complication is (as you can guess from the title), the PIE security feature being enabled.

Enumeration

Being a beginner challenge, we get both the binary `vuln` and source code `vuln.c`.

Running `file` and `checksec` on `vuln` confirms what we already gathered from the title:

```
demo-picoctf@webshell:~$ file ./vuln
./vuln: ELF 64-bit LSB pie executable, x86-64, version 1 (SYSV), dynamically linked, interpreter /lib64/ld-linux-x86-64.so.2, BuildID[sha1]=0072413e1b5a0613219f45518ded05fc685b680a, for GNU/Linux 3.2.0, not stripped
demo-picoctf@webshell:~$ checksec ./vuln
[*] '/home/demo-picoctf/vuln'
Arch:             amd64-64-little
RELRO:            Full RELRO
Stack:            Canary found
NX:               NX enabled
PIE:              PIE enabled
SHSTK:            Enabled
IBT:              Enabled
Stripped:         No
demo-picoctf@webshell:~$
```

Next we peer at the source code in `vuln.c`. The first thing that caught my eye is the `win()` function. Sure enough it looks to be what will give us our flag:

```
int win() {
    FILE *fptr;
    char c;

    printf("You won!\n");
    // Open file
    fptr = fopen("flag.txt", "r");
    if (fptr == NULL)
    {
        printf("Cannot open file.\n");
        exit(0);
    }

    // Read contents from file
    c = fgetc(fptr);
    while (c != EOF)
    {
        printf ("%c", c);
        c = fgetc(fptr);
    }

    printf("\n");
    fclose(fptr);
}
```

So what does `main()` have in store for us? Well after setting up a signal handler for segmentation faults it straight up gives us the address of the `main` function:

```
printf("Address of main: %p\n", &main);
```

Next, it gives us the chance to enter in an address (presumably for a function) we want to jump to:

```

unsigned long val;
printf("Enter the address to jump to, ex => 0x12345: ");
scanf("%lx", &val);
printf("Your input: %lx\n", val);

void (*foo)(void) = (void (*)(void))val;
foo();

```

Looks simple enough: Find the address of `win()` in the binary -> Input it into the program -> Receive flag!

Walkthrough

If we're going to jump to `win()` first we need to find its address. Unfortunately because of the nature of PIEs (Position Independent Executables), `win()`'s address will be in constant flux -- we will need to approach with something that is NOT constantly changing. I inspected the binary with `objdump -d` to look at the offsets of the instructions in the file:

```

00000000000012a7 <win>:
 12a7:    f3 0f 1e fa      endbr64
 12ab:    55               push    rbp
 12ac:    48 89 e5         mov     rbp, rsp
 12af:    48 83 ec 10      sub     rsp, 0x10
 12b3:    48 8d 3d 74 0d 00 00 lea     rdi, [rip+0xd74]
 12ba:    e8 41 fe ff ff   call    1100 <puts@plt>
 12bf:    48 8d 35 71 0d 00 00 lea     rsi, [rip+0xd71]
 12c6:    48 8d 3d 6c 0d 00 00 lea     rdi, [rip+0xd6c]
 12cd:    e8 9e fe ff ff   call    1170 <fopen@plt>
 12d2:    48 89 45 f8      mov     QWORD PTR [rbp-0x8], rax
 12d6:    48 83 7d f8 00   cmp     QWORD PTR [rbp-0x8], 0x0
 12db:    75 16            jne     12f3 <win+0x4c>
 12dd:    48 8d 3d 5e 0d 00 00 lea     rdi, [rip+0xd5e]
 12e4:    e8 17 fe ff ff   call    1100 <puts@plt>
 12e9:    bf 00 00 00 00   mov     edi, 0x0
 12ee:    e8 9d fe ff ff   call    1190 <exit@plt>
 12f3:    48 8b 45 f8      mov     rax, QWORD PTR [rbp-0x8]
 12f7:    48 89 c7         mov     rdi, rax
 12fa:    e8 41 fe ff ff   call    1140 <fgetc@plt>
 12ff:    88 45 f7         mov     BYTE PTR [rbp-0x9], al
 1302:    eb 1a            jmp     131e <win+0x77>
 1304:    0f be 45 f7      movsx   eax, BYTE PTR [rbp-0x9]
 1308:    89 c7            mov     edi, eax
 130a:    e8 e1 fd ff ff   call    10f0 <putchar@plt>
 130f:    48 8b 45 f8      mov     rax, QWORD PTR [rbp-0x8]
 1313:    48 89 c7         mov     rdi, rax
 1316:    e8 25 fe ff ff   call    1140 <fgetc@plt>
 131b:    88 45 f7         mov     BYTE PTR [rbp-0x9], al
 131e:    80 7d f7 ff      cmp     BYTE PTR [rbp-0x9], 0xff
 1322:    75 e0            jne     1304 <win+0x5d>
 1324:    bf 0a 00 00 00   mov     edi, 0xa
 1329:    e8 c2 fd ff ff   call    10f0 <putchar@plt>
 132e:    48 8b 45 f8      mov     rax, QWORD PTR [rbp-0x8]
 1332:    48 89 c7         mov     rdi, rax
 1335:    e8 d6 fd ff ff   call    1110 <fclose@plt>
 133a:    90               nop
 133b:    c9               leave
 133c:    c3               ret

000000000000133d <main>:

```

Presented in the same order as in the source code, we see the offset of `win()` at `0x12a7` and `main()` at `0x133d`. Remember: **offsets ≠ addresses**. Offsets are tied to the *file* on disk the

instructions are in, while addresses are where those instructions are loaded in memory at runtime. Basically, the offsets are important because they will remain constant unlike the addresses. We can calculate the distance between the two functions as `0x96` bytes (`0x133d` - `0x12a7`). Now we can use whatever the printed address of `main()` is to calculate, and then input, the address of `win()` .

Solution

So the steps are:

1. Calculate the **distance** between `win()` and `main()`
2. Run program and subtract (because `win()` comes before `main()`) the distance (`0x96`) from `main()` 's current address
3. Input the new address and receive flag!